

COVER_BINS

Overview

Cover_bins collects coverage information for a group of coverage bins during the simulation run. It contains a set of predefined counters, each incrementing upon specified event. For each counter, users may set the maximum and minimum limits. When ALL counters reach their minimum limits, the **min_limit** access event toggles. When ONE of the counters reaches its maximum limit, the **max_limit** access event toggles. All coverage information can be accessed dynamically in order to tune test the generation process and to define additional constraints and length of the simulation run.

Syntax

```
Cover_bins (#cover_num, ssize) InstName ();
```

Parameters		
cover_num		Maximum number of coverage items allowed. Default: 0
ssize		String size for the name of cover item. Default: 20
Access Tasks		
Name	Arguments	Description
reset	No	Resets coverage counters and maximum & minimum limits
add	bin_name min_limit max_limit	- Name of the coverage bin (with length <= ssize) - Lower limit for the counter. Default: 0 - Upper limit for the counter. Default: 32'hFFFF_FFFF Use "add" access task to add coverage bins to the vc_cover coverage group. Example: <InstName>.add("idle_cs_num", 10, 30) – adds coverage bin with the name "idle_cs_num", lower limit e.q. to 10 and upper limit equals to 30
change	bin_name count min_limit max_limit	- Name of the coverage bin (with length <= ssize) - Counter number for the coverage item - Lower limit for the counter. Default: 0 - Upper limit for the counter. Default: 32'hFFFF_FFFF Use "change" access task to edit functional item data. Example: <InstName>.edit("idle_cs_num", 0, 10, 40) – resets counter and change max. limit from 30 to 40 for the coverage item "idle_cs_num"
sample	Cover_name	Use "sample" task to increment coverage bin counter and update coverage information. Example: <pre>always @(posedge clk) if (a_reg == 0) cov.sample("a_reg = 0");</pre>
report	No	Generates coverage report in the tabular format. See below for the report example.
Access Events		
covered		"Fires" when all vc_cover counters exceed their lower limits. Can be monitored externally to define length of the test case execution based on accumulated coverage information.
exceeded		"Fires" when anyone of counters exceed predefined maximum limit. Maximum limits can be used as additional test-specific constraints. When it fires, "Exceeded_name" access register provides the name of a cover bin exceeding the maximum limit.
Access Variables		
total_cov		Provided a constantly updated value of total coverage (in percents). After a "covered" event, it has to be equal to 100 %.
exceeded_name		Name of a coverage bin with the counter exceeding a predefined maximum value.

Control Variables	
Display_status	Set it to "1" to get more informational messages. Default: 0

Verilog example

The following example shows how coverage information controls the length of the simulation run. By monitoring "covered" event, test case is running until a pre-defined coverage is accumulated. By monitoring "exceeded" event, test case exits upon the first violation.

```
module test;

reg clk; initial clk = 0;
reg [3:0] a_reg, b_reg, c_reg, d_reg;

always #10 clk <= ~clk;

always @(posedge clk) begin
    a_reg <= ($random)%4;
    b_reg <= ($random)%4;
    c_reg <= ($random)%4;
    d_reg <= ($random)%4;
end

//-----
vc_cover #4 cov ();
initial begin
    cov.display_status = 0;
    cov.add("a_reg = 0", 10, 40);
    cov.add("b_reg = 1", 10, 40);
    cov.add("c_reg = 2", 10, 40);
    cov.add("d_reg = 3", 10, 40);
end
//-----
always @(posedge clk) begin
    if (a_reg == 0) cov.sample("a_reg = 0");
    if (b_reg == 1) cov.sample("b_reg = 1");
    if (c_reg == 2) cov.sample("c_reg = 2");
    if (d_reg == 3) cov.sample("d_reg = 3");
end

initial forever begin
    #1000 cov.report;
end

// Exit on first maximum violation
always @(cov.exceeded) begin
    $display ("MAXIMUM reached for '%s'", cov.exceeded_name);
    $finish;
end

// Run till all functional coverage is collected; then exit
always @(cov.covered) begin
    $display("[%0t] Info: Stimulus coverage reached, test may exit", $time);
    cov.report;
    $finish;
end

//If functional coverage is not collected , use timebomb
```

```

initial begin
    $dumpvars;
    #10000 $finish;
end

endmodule

```

Coverage Report Example

Coverage Report <test.cov.report>				
Cover name	Count	Min_limit	Max_limit	Coverage
a_reg = 0	24	10	40	100 percent
b_reg = 1	12	10	40	100 percent
c_reg = 2	22	10	40	100 percent
d_reg = 3	5	10	40	50 percent
Total Coverage:		87 percent		